LEANFM · A COMPACT ACCOUNT

LeanFM in Brief

Event streams, recursive productions and CTL: a temporal-logic method for software engineering

Stream	Behavior is a stream of timestamped events, linked backward by *cause* and *prev*.
Grammar	Productions say which streams are legal. Recursion is the only control flow.
CTL	Formulas attached to productions say what must, or may, happen next.
Picture	Every trace, witness and counterexample is drawn as an interaction diagram.

Rob Fielding · checked with LeanFM 4788b14 2026-10-28

Contents

1. The method on one page
2. Events: what the stream records
3. Productions: recursion as control flow
4. Interaction diagrams
5. CTL attached to productions
6. Time
7. Checking: from productions to verdicts
8. Witnesses and counterexamples
9. Finding a bug
0. Past and future
1. Working this way

Appendices: A. Notation · B. JSON forms · C. Reproducing this book

> **How to read the figures**
> Every diagram, verdict and formula reading in this book is **generated**. The example specifications are compiled into task models and passed to LeanFM's own `check_ctl` and `check_trace` tools, the same code the LeanFM MCP server runs. Each figure carries a provenance line giving the tool and its verdict. Rebuilding the book reruns every check, and the build fails if a verdict stops matching the text.

1 The method on one page

Specify only what can be observed. Say which observations may follow which. Then ask, at each point, what the future must hold and what it can still hold.

A system is a set of **actors** exchanging **messages**. Every message that crosses an actor boundary is an **event**, and the run of a system is a **stream** of events. LeanFM specifies that stream and nothing behind it: no internal variables, no object graphs, no private state that an observer could not see.

1 · Events

Each event has an id, a timestamp, sender, receiver, message and fields. It points back to its cause and its prev.

2 · Productions

A grammar over events. Choice, sequence and calls, with **recursion as the only loop**. `when` clauses relate fields and times.

3 · CTL

Each production carries formulas such as `AG EF terminal`: from here, finishing always stays possible.

4 · Diagrams

A trace is drawn as an interaction diagram: lifelines, arrows, cause brackets, production frames.

5 · Checking

Productions compile to a finite transition system. CTL is decided on it, with a witness or counterexample path.

6 · Monitoring

Recorded streams are checked against the same productions: links, timestamps and acceptance.

The two directions of time are handled differently, on purpose:

- **The past is data.** It is already in the stream. An event names the event that caused it and the event before it on its thread, so any fact about history is found by following links backward. There are no past-time operators.
- **The future is branching.** The grammar allows alternatives, so from any point there is a *tree* of legal continuations. CTL is the logic of that

tree: `E` means some continuation, `A` means every continuation.

The result is a specification of manageable size that is easy to draw and check, and that can be handed to a person or an LLM as the complete, durable statement of what a system must do. The running example throughout is a login session:

```
-- A client logs in, browses, and leaves.
protocol Session
actors Client Server

Session ::= Login
  ctl [ not get AW welcome ]
  ctl [ not get AU welcome ]
  ctl EF bye
  ctl AG EF terminal
  ctl get ==> AF (page or notfound)

Login ::= l: Client -> Server : login(user) ;
          ( Server -> Client : welcome(sid) cause l
              when valid(user) ;
            Work(sid)
          | Server -> Client : denied() cause l
              when not valid(user) ;
            Login )

Work(s) ::= Request(s) ; Work(s)
          | o: Client -> Server : logout(s) ;
            Server -> Client : bye() cause o
          | x: Server -> Client : expired(s)
              when x.timeAt - x.prev.timeAt >= 30min
  ctl AG EF terminal
  ctl AF terminal

Request(s) ::= g: Client -> Server : get(s, path) ;
               ( Server -> Client : page(s, body) cause g
                   when within(g, 2s)
               | Server -> Client : notfound(s) cause g
                   when within(g, 2s) )
  ctl AF terminal
```

The rest of the book explains this file one feature at a time.

2 Events: what the stream records

An event is one observation at one instant. Everything else, including state, duration, sessions and causality, is computed from events.

The event envelope is deliberately small:

`id`
: Unique within the stream.

`timeAt`
: Milliseconds on one observer's monotonic clock. Only differences matter.

`task`
: The conversation the event belongs to, such as one session. Many tasks interleave in one stream.

`src`, `dst`, `message`
: Who sent what to whom. Actors may be instances of a role: `Client(ann)` plays `Client`.

`prevIds`
: The immediately preceding event(s) on the same thread. A fork gives several events that share a prev; a join lists several prevs.

`causeIds`
: The event(s) this one answers or finishes. A response names its request; an end names its start.

```
{ "id": "b2", "prevIds": ["b1"], "causeIds": ["b1"], "task": "bob", "src":
"Server", "dst": "Client(bob)", "message": "denied", "timeAt": 150 }
```

Two backward links, two jobs

prev is *order*. Following prev links reconstructs each conversation from an interleaved stream, without trusting arrival order. Two events are **concurrent** when neither can reach the other through the links.

cause is *bracketing*. A request and the response that names it enclose everything in between, like a call and its return. Brackets nest, so a flat stream becomes a tree of calls. Because the link names a specific event, the right request is matched even when many are in flight.

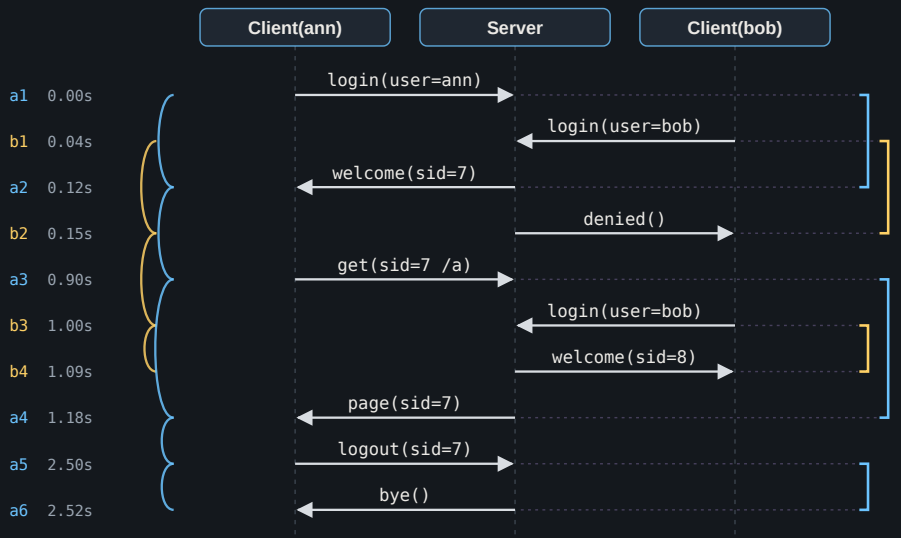


Figure 2.1 Two sessions interleaved in one stream. The left threads are prev links: blue for ann's task, gold for bob's. The right brackets are cause links, each joining a reply to its request. Bob's second login is bracketed inside ann's `get … page`: nested in time, but in a different task.

> LeanFM `check_trace`: **admissible** — 10 events over 2 task(s); 24 concurrent pair(s) · task ann: accepted=true, reached terminal=true; task bob: accepted=true, reached terminal=false

From the same stream, `check_trace` also returns the happens-before graph. Solid arrows are prev; dashed arrows are cause. Ann's and bob's events are unordered with respect to each other: LeanFM counts **24** concurrent pairs (for example a1‖b1, a1‖b2, a1‖b3, a1‖b4).



Figure 2.2 The causal graph exactly as `check_trace` emits it, rendered from its Mermaid output.

Timestamps

A timestamp is meaningful mainly in a *difference along a link*. The latency of a request is `reply.timeAt − request.timeAt`, with the reply found through its cause link. An idle gap is `e.timeAt − e.prev.timeAt`. The one rule the stream itself must obey is that no prev or cause link runs backward in time. Global order across tasks is not needed and is not claimed.

3 Productions: recursion as control flow

A protocol is a language whose letters are events. Productions say which words are legal.

A production names an interaction. Its body is built from five things:

`l: Client -> Server : login(user)`	an event; the optional label `l` lets later events refer to it
`a ; b`	sequence: `a`, then `b`
`a | b`	choice: exactly one branch happens
`Work(sid)`	call another production, passing values bound earlier
`skip`	nothing happens (an empty branch)

An event may add `cause l`, meaning "this event answers `l`". That emits the cause link and opens or closes a bracket. It may also add `when …`, a predicate over fields, parameters and timestamps (Chapter 6). A choice is decided by whichever actor sends the first event that tells the branches apart. In `Login` the Server decides, by answering `welcome` or `denied`. `Work` breaks that rule on purpose: the Server's `expired` races the Client's next `get` or `logout`. The idle-time bound is what settles the race. A realizability check would flag the race, and LeanFM does not yet perform that check.

There is no loop construct

There is no `while`, no `repeat`, no Kleene star and no state variable. Repetition is recursion, in three shapes:

Tail recursion is a loop.
`Work(s) ::= Request(s) ; Work(s) | …` keeps serving requests. The tail call replaces the current instance, so nothing accumulates.

Choice plus recursion is a loop with an exit.

`Login` retries after `denied` by calling itself and leaves through `welcome`. The exit condition is simply the other branch.

A non-tail call nests.

A call between a request and its reply, like `Upstream` below, opens a bracket inside a bracket. The stream shows the call stack.

```
-- A non-tail call nests one bracket inside another.
protocol Fetch
actors Client Proxy Server

Fetch ::= g: Client -> Proxy : get(path) ;
          Upstream(path) ;
          Proxy -> Client : page(body) cause g
  ctl AF terminal
  ctl EF content

Upstream(path) ::= u: Proxy -> Server : fetch(path) ;
                   Server -> Proxy : content(body) cause u
                 | skip
```

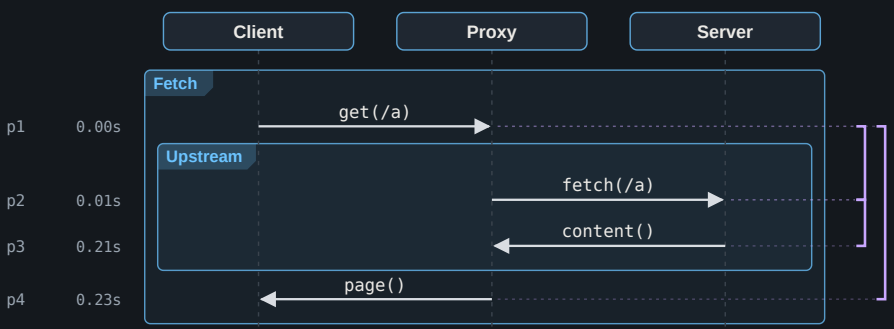


Figure 3.1 The non-tail call `Upstream` sits inside `Fetch`'s bracket. The inner cause bracket (`fetch` … `content`) nests inside the outer one (`get` … `page`). `fetch` also names `get` as its cause, which gives the third bracket.

> LeanFM `check_trace`: **admissible** — 4 events over 1 task(s); 0 concurrent pair(s)

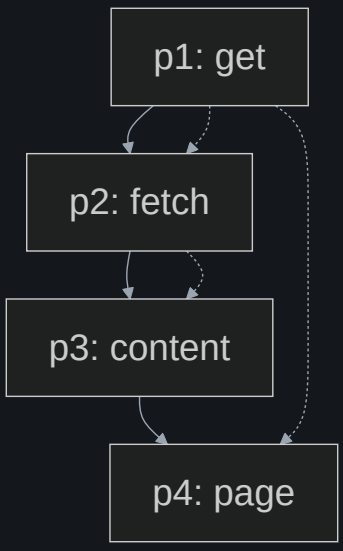


Figure 3.2 The same stream's causal graph from `check_trace`.

Why recursion? It makes loops *grammatical*, not operational. Every iteration is a production instance with a beginning and an end, which a diagram can frame and a formula can talk about. Tail recursion keeps the transition system finite. Bracketed recursion is exactly the nesting that cause links already express. Unbounded non-tail recursion is the one shape a finite checker cannot represent; the companion compiler rejects it, and a model that needs it must bound its depth.

4 Interaction diagrams

The one picture that matters. A trace, a witness and a counterexample are all streams, so all of them are interaction diagrams.

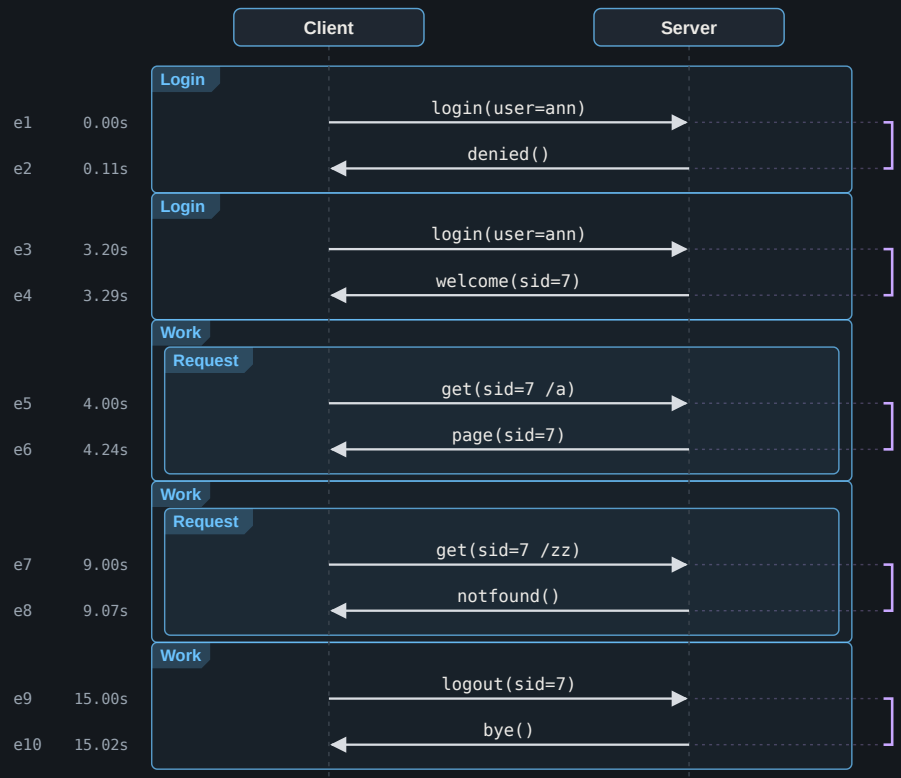


Figure 4.1 One session, parsed by the productions of Chapter 1: a denied login, a retry, two requests and a goodbye.

> LeanFM `check_trace`: **admissible** — 10 events over 1 task(s); 0 concurrent pair(s) · task Session accepted, reached terminal=true

How to read it:

- **Lifelines** are actors. **Rows** are events in stream order. The left column shows each event's id and `timeAt`.
- An **arrow** is one message from sender to receiver, labeled with the message and its fields.

- A **bracket** on the right joins an event to the event it names as its cause. Nested brackets are nested calls.
- A **frame** is one production instance in the parse. A tail call is drawn as the next frame at the same level (`Login`, `Login`; `Work`, `Work`, `Work`), because tail recursion is a loop. A non-tail call (`Request` inside `Work`) is drawn nested.
- Red rows are violations. A dashed amber frame marks the part of a counterexample that **repeats forever**.

Frames are computed, not drawn: the trace is matched against the productions, and every production instance in the successful parse becomes a frame. A trace that does not parse has no frames, and its first unparseable event is the error to report.

5 CTL attached to productions

The grammar generates a tree of possible futures. CTL quantifies over that tree. Attaching a formula to a production says where the question is asked.

Every temporal operator in CTL pairs a **path quantifier** with a **temporal word**:

Formula	Reading	Typical use
`EF p`	possibly, eventually p	reachability, with a witness
`AF p`	on every future, eventually p	termination, response
`AG p`	on every future, at every point, p	safety invariant
`EG p`	on some future, p forever	a path that can stay good
`AX p`, `EX p`	every / some next step satisfies p	one-step consequence
`[p AU q]`	every path keeps p until q, and q happens	constrained progress
`[p EU q]`	some path keeps p until q, and q happens	possible progress
`[p AW q]`	every path keeps p until q, or keeps p forever	"not before"
`[p EW q]`	some path keeps p until q, or keeps p forever	a path that can wait
`never p`	`AG not p`	forbidden behavior
`p ==> q`	`EF p` and `AG (p => q)`	non-vacuous implication

Until is written infix, with its quantifier attached: `[p AU q]`, never `A[p U q]`. The second form looks like an LTL formula placed inside a quantifier, but CTL has no path formulas of its own. `AU` is one operator, just as `AG` and `EF` are, and this is the form LeanFM uses in its JSON (`{"AU": [p, q]}`).

Atoms are events

A message name, like `get`, holds at the point just after a `get` was delivered. `terminal` holds when the production has finished. A finished run stutters in place forever, so every path is infinite and `AF` and `AG` have their textbook meaning. Atoms never mention hidden state: if a requirement needs a fact no event reveals, the fix is to add the event.

Scope: what an attached formula talks about

> **The attachment rule**
> A `ctl φ` line under production `P` must hold at the moment any instance of `P` begins, over every continuation of that instance: its own events, everything it calls, and everything it tail-calls, until the instance is finished. For `Request`, finished means its reply. For `Work`, which tail-calls itself, it means the end of the session.

This puts each formula where it is true. "Every request is answered" is `AF terminal` on `Request`, a two-event fact checked on a three-state model. On `Session` the same idea needs the longer `get ==> AF (page or notfound)`. Local formulas also give short counterexamples.

Idioms

`AG EF terminal`	no trap: finishing is always still possible
`AF terminal`	always finishes (often *too strong*, see Chapter 8)
`[ not x AW y ]`	no `x` before `y`
`EF x`	`x` is reachable; the witness is a usage scenario
`never x`	`x` is forbidden; pair it with the positive `AG` it protects
`x ==> AF y`	every `x` is followed by `y`, and `x` can actually happen

LeanFM reads the Session's first formula aloud as: *on every path, (it is not the case that get was just delivered) holds unless welcome was just delivered (welcome was just delivered need not happen)*. Its negation,

which is what a counterexample must show, reads: *on some path, (it is not the case that welcome was just delivered) holds until (get was just delivered and (it is not the case that welcome was just delivered)) (and (get was just delivered and (it is not the case that welcome was just delivered)) does happen)*.

6 Time

CTL stays untimed. Time enters through timestamps on events and `when` clauses on productions.

A `when` clause can read the fields of labeled events, the parameters of the production, and the `timeAt` of any event reachable backward through cause and prev. Two patterns cover most timing requirements:

Bounded response

`Server -> Client : page(s, body) cause g when within(g, 2s)`, where `within(g, d)` abbreviates `this.timeAt − g.timeAt ≤ d`. Together with `AF terminal` on `Request`, this gives bounded liveness: a reply always comes, and comes within two seconds.

Timeout as a branch

`x: Server -> Client : expired(s) when x.timeAt − x.prev.timeAt ≥ 30min`. A timeout is not a hidden timer. It is an event, a branch of the choice, enabled by the idle gap since the previous event on this thread.

Because the bound lives on the event that ends the interval, and the interval's start is named by a link, a duration is always unambiguous, even with many overlapping requests.

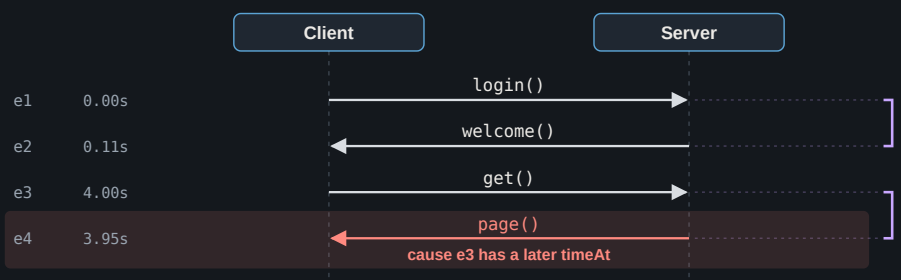


Figure 6.1 A reply stamped 50 ms *before* its request. The stream contradicts itself, whatever the productions say.

> LeanFM `check_trace`: **inadmissible** — 4 events over 1 task(s); 0 concurrent pair(s) · e4: prev e3 has a later timeAt (4000 > 3950)

What is checked today

LeanFM's `check_trace` enforces that ids are unique, that links point to earlier events, that no link runs backward in time, that links are acyclic, and that each task is accepted by its compiled productions. When the model checker builds the futures tree, it treats `when` clauses as free choices: `valid(user)` may go either way at every login. Evaluating `when` clauses, including the timing bounds, against a recorded stream is part of the method but not yet part of the tools.

7 Checking: from productions to verdicts

Productions compile to a finite transition system. CTL is decided on it by fixed-point iteration, and the answer comes with a path.

Compiling a production inlines every call. A tail call back into a production that is already being expanded becomes an edge back to that production's start, so recursion becomes a cycle. Equivalent states are then merged. `Work` becomes:

from	event	to
s0 ▸	`Client → Server : get`	s1
s0 ▸	`Client → Server : logout`	s2
s0 ▸	`Server → Client : expired`	s3 ■
s1	`Server → Client : page`	s0
s1	`Server → Client : notfound`	s0
s2	`Server → Client : bye`	s3 ■

▸ initial state · ■ terminal state. The tail call `Work(s)` after `page` or `notfound` is the edge back to `s0`.

That table *is* the task model sent to LeanFM: states, an initial state, terminal states, and transitions labeled with message, sender and receiver. LeanFM builds its reachable nodes (a state plus the message that reached it, so that atoms like `get` can be evaluated) and computes each formula by least or greatest fixed point. A *universal* claim that fails comes back with a counterexample path. An *existential* claim that holds comes back with a witness. When the path loops, LeanFM says where it loops back.

Results for the session protocol

Production	Formula	LeanFM verdict	Evidence
Session	`[ not get AW welcome ]`	**proved**	all 10 nodes
Session	`[ not get AU welcome ]`	**disproved**	counterexample: login · denied ↺
Session	`EF bye`	**proved**	witness: login · welcome · logout · bye
Session	`AG EF terminal`	**proved**	all 10 nodes
Session	`get ==> AF (page or notfound)`	**proved**	witness: login · welcome · get
Work	`AG EF terminal`	**proved**	all 7 nodes
Work	`AF terminal`	**disproved**	counterexample: get · page ↺
Request	`AF terminal`	**proved**	all 4 nodes

Production	Formula	LeanFM verdict	Evidence
Fetch	`AF terminal`	**proved**	all 5 nodes
Fetch	`EF content`	**proved**	witness: get · fetch · content

Two formulas are disproved, and both failures teach something. They are the subject of the next chapter.

8 Witnesses and counterexamples

Evidence is a stream, so evidence is an interaction diagram. A requirement is read through its witness and its counterexample.

A witness is a usage scenario



Figure 8.1 `EF bye` on `Session`: the shortest path to a clean goodbye.

> LeanFM `check_ctl`: **proved** — it is possible that eventually bye was just delivered

Every `EF` formula yields one of these. Collected together, they are the protocol's documentation by example, and they cannot go stale, because they are regenerated from the productions.

A lasso is a forever

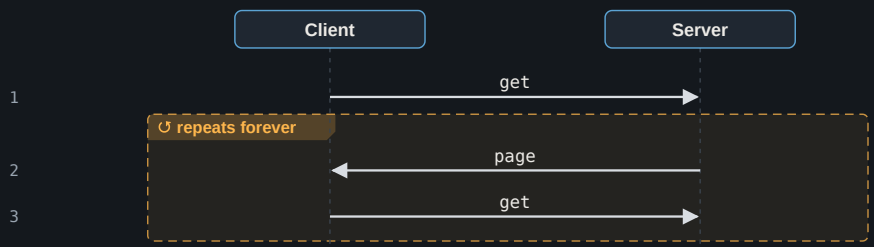


Figure 8.2 `AF terminal` on `Work` is disproved. After one `get`, the client may answer every `page` with another `get`, forever.

> LeanFM `check_ctl`: **disproved** — on every path, eventually the task has ended

The checker is right, and the counterexample forces a decision: is endless browsing acceptable? For a web session it is, so the real require-

ment is the weaker `AG EF terminal` (finishing always stays possible), which is proved. Had the answer been no, the fix would have been in the grammar, for example a request limit passed as a parameter.

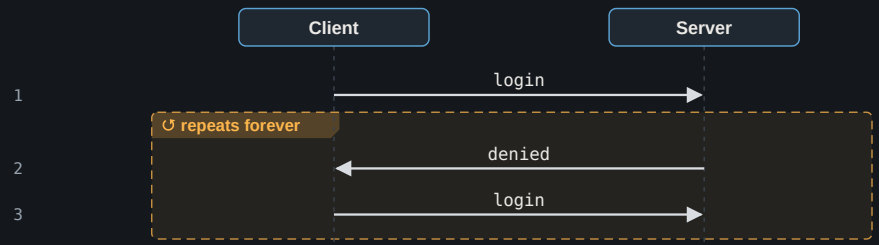


Figure 8.3 `[ not get AU welcome ]` is disproved by a client that is denied forever. Strong until demands that `welcome` actually happens.

> LeanFM `check_ctl`: **disproved** — on every path, (it is not the case that get was just delivered) holds until welcome was just delivered (and welcome was just delivered does happen)

The intent was only "no request before a welcome". That is weak until, `[ not get AW welcome ]`, which is proved. The difference between `AU` and `AW` is exactly the forever path in the figure.

9 Finding a bug

Change one call in the grammar. The formulas catch it before any code exists, and the trace check catches it after.

Suppose a server is written so that a denied login falls through into the work loop: one call changed, `Login` to `Work(sid)`.

```
Login ::= l: Client -> Server : login(user) ;
          ( Server -> Client : welcome(sid) cause l
              when valid(user) ;
            Work(sid)
          | Server -> Client : denied() cause l
              when not valid(user) ;
            Work(sid) )
```

Before any code exists, the Session's first formula fails on the edited grammar:

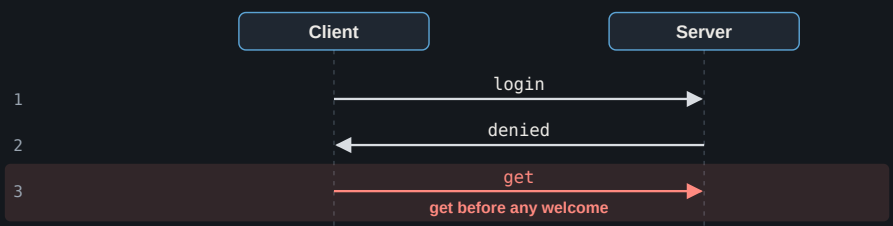


Figure 9.1 Counterexample to `[ not get AW welcome ]` on the edited grammar: a request is served after a denial.

> LeanFM `check_ctl`: **disproved** — on every path, (it is not the case that get was just delivered) holds unless welcome was just delivered (welcome was just delivered need not happen)

After code exists, the same mistake shows up in a recorded stream. Checked against the *correct* productions, the trace is rejected at the first event the grammar does not allow, and LeanFM reports what it expected instead:

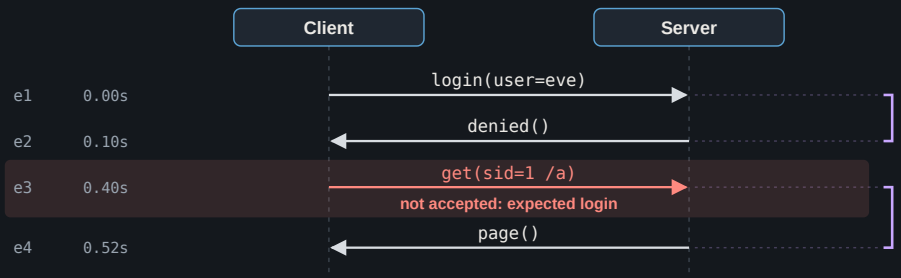


Figure 9.2 A stream from the buggy server, checked against the specification.

> LeanFM `check_trace`: **inadmissible** — 4 events over 1 task(s); 0 concurrent pair(s) · task Session: event e3 is not accepted by the task's transitions (expected: login)

The same specification produced both results: a counterexample from the model before implementation, and a rejection from the stream after it.

10 Past and future

The stream remembers; CTL anticipates.

Past: follow links

The history behind an event is the set of events reachable backward through cause and prev. It is a fixed fact, a single history, so no quantifier is needed. `when` clauses read it: `x.prev.timeAt`, `within(g, 2s)`, "this `sid` equals the one issued by the `welcome` that caused this login to succeed".

Future: quantify paths

The future after a prefix is a tree, because productions branch. `A` and `E` range over its branches; `F`, `G`, `X`, `U` and `W` range along a branch. Possibility (`EF`) and necessity (`AF`, `AG`) stay distinct, and LTL cannot express that distinction.

Keeping the past as data means a requirement never has to restate history as hidden state. Two runs that look identical on the stream cannot be told apart by the specification. If the future must depend on something, that something must have been an event.

The same division gives runtime monitoring. As a stream arrives, each task's prefix is parsed by its productions, and the parser's position is a node of the transition system. Safety (`AG`, `never`) fails on the first bad event, as in Figure 9.2. A possibility (`EF`) can be found impossible early, as soon as the current node has no path to it. An obligation (`AF`) is still open while the run continues, and the monitor reports what is pending. Today `check_trace` does the acceptance part for a complete recorded stream; running it incrementally on a live stream is the same computation applied event by event.

11 Working this way

Requirements are the public contract. Everything else is implementation detail and can be regenerated.

1. **Name the actors and messages that cross boundaries.** If no event reveals a fact, it is not a requirement yet.
2. **Write productions.** Use recursion for every repetition. Put a `cause` on every reply. Bind values once and pass them as parameters.
3. **Attach the positive invariant first**, usually an `AG`. Then the forbidden behavior, as `never` or `[not x AW y]`. Then the possibilities, as `EF`.
4. **Check.** Read every witness as a scenario and every counterexample as a question. Fix either the grammar or the formula, whichever was wrong.
5. **Freeze what others depend on.** Once a client relies on a production, it is public. Persistent implementation choices (storage layout, byte encodings) go in a separate implementation specification.
6. **Implement in one pass** from the checked specifications, then check recorded streams against the same productions.

This suits working with an LLM. Questions in a requirements conversation should either pin down public behavior or record a persistent implementation decision. The diagrams and verdicts are the review artifacts, and git is where they are accepted. When the specification is complete, the chat history can be thrown away: the book, the productions and their checks are the durable statement of what to build.

A Notation

```
spec        ::= 'protocol' Name  'actors' Name+  production+
production  ::= Name ['(' params ')'] '::=' body  ('ctl' formula)*
body        ::= seq ('|' seq)*
seq         ::= item (';' item)*
item        ::= event | call | 'skip' | '(' body ')'
event       ::= [label ':'] Actor '->' Actor ':' msg '(' fields ')'
                ['cause' label] ['when' expr]
call        ::= Name ['(' args ')']

formula     ::= f ['=>' formula | '==>' formula]
f           ::= g ('or' g)*        g ::= u ('and' u)*
u           ::= 'not' u | 'never' u
              | ('AG'|'AF'|'EG'|'EF'|'AX'|'EX') u
              | '[' formula ('AU'|'EU'|'AW'|'EW') formula ']'
              | '(' formula ')' | 'true' | 'false' | 'terminal' | msg
```

- Comments begin with `--`. Newlines are not significant.
- A `cause` label must name an earlier labeled event in the same production body. This is checked statically.
- A choice must be decided by one actor: the first distinguishing event of every branch has the same sender.
- A call is a **tail call** when nothing follows it in its production. Recursion must be through tail calls, or bounded.
- `a || b` (interleaving, order kept within each side) belongs to the full LeanFM grammar. The companion compiler used for this book does not yet expand it.

B JSON forms

These are the exact shapes passed to LeanFM's MCP tools. They are also what an implementation's logs should contain.

Event (`check_trace`)

```
{ "id": "b2", "prevIds": ["b1"], "causeIds": ["b1"], "task": "bob", "src":
"Server", "dst": "Client(bob)", "message": "denied", "timeAt": 150 }
```

Formula (`check_ctl`)

`[ not get AW welcome ]` becomes

```
{ "AW": [{ "not": { "after": "get" } }, { "after": "welcome" }] }
```

`get ==> AF (page or notfound)` becomes

```
{ "stronglyImplies": [{ "after": "get" }, { "AF": { "or": [{ "after": "page"
}, { "after": "notfound" }] } }] }
```

Task model: the compiled `Work` production

```
{"id": "Work", "initial": "s0",
 "states": ["s0","s1","s2","s3"], "terminals": ["s3"],
 "transitions": [
  {"src":"s0","dst":"s1","message":"get","from":"Client","to":"Server"},
  {"src":"s0","dst":"s2","message":"logout","from":"Client","to":"Server"},
  {"src":"s0","dst":"s3","message":"expired","from":"Server","to":"Client"},
  {"src":"s1","dst":"s0","message":"page","from":"Server","to":"Client"},
  {"src":"s1","dst":"s0","message":"notfound","from":"Server","to":"Client"},
  {"src":"s2","dst":"s3","message":"bye","from":"Server","to":"Client"}]}
```

C Reproducing this book

```
~/code/leanFM     LeanFM (git: origin/main)
~/code/seLean     this book
  examples/session.lfm   the running example
  examples/proxy.lfm     nested brackets
  examples/traces.mjs    the event streams
  tools/lfm.mjs          parser, production compiler, frame matcher
  tools/seq.mjs          interaction-diagram renderer
  Gen.lean               runs check_ctl, check_trace
  build.mjs              checks, figures, PDF

./build.sh      # lake build gen, then node build.mjs
```

`Gen.lean` calls `LeanFM.Logic.run`, the function the LeanFM MCP server dispatches its logic tools to, so no server needs to be running. The build asserts every verdict that the text relies on. Changing a production so that a claim in the book becomes false makes the build fail.